

AI Lab # 2

This lab explores uninformed search strategies by implementing Breadth-First Search to systematically find solutions in state-space problems.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 01 August, 2026

Today's Agenda

01

Uninformed Search

02

What is BFS?

03

Algorithm Steps

04

Examples and Lab Task

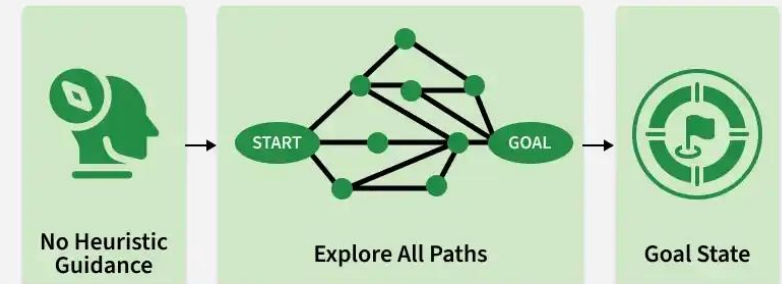
05

8-Puzzle Problem Overview

Uninformed Search

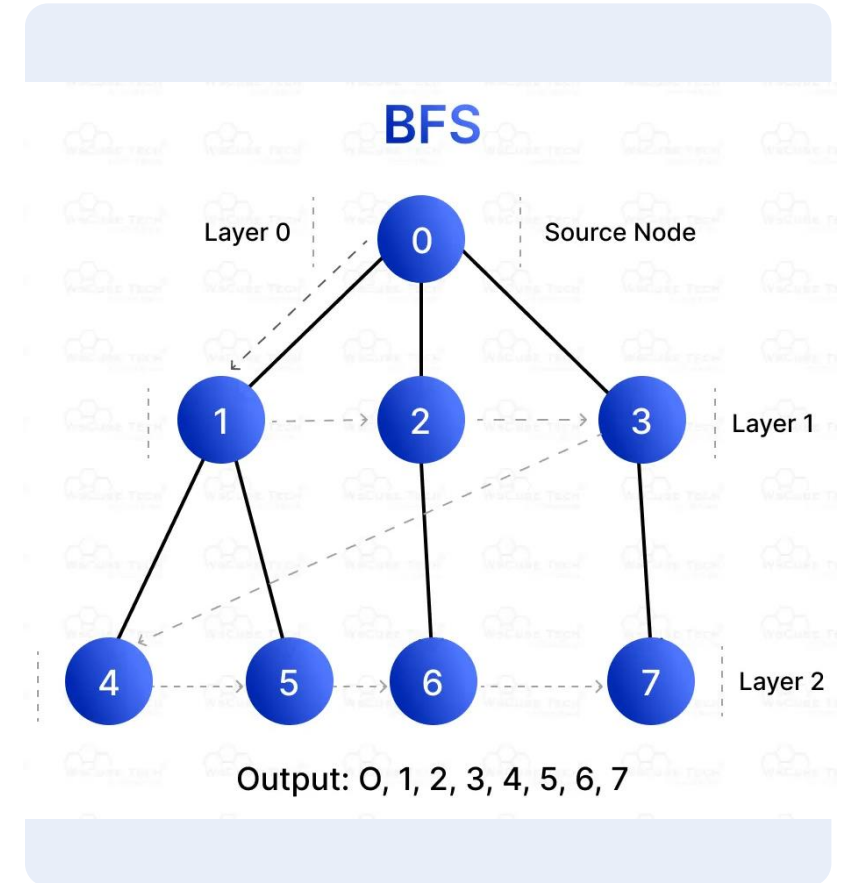
- Uninformed search algorithms also called blind search algorithms.
- They have no additional information about goal location except the problem definition.
- They explore the search space blindly.
- Examples are BFS, DFS, UCS

Uninformed Search in Artificial Intelligence



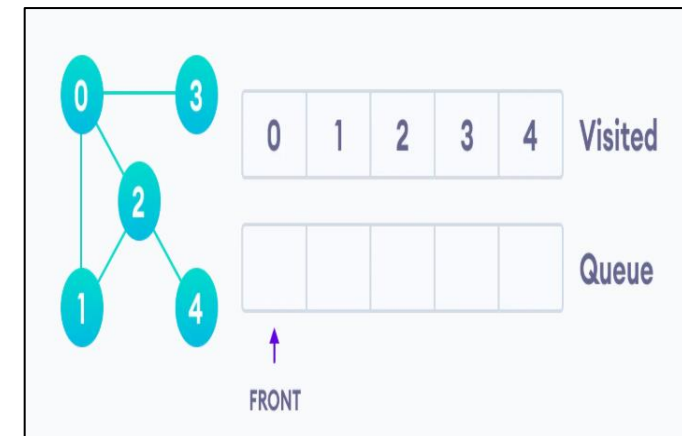
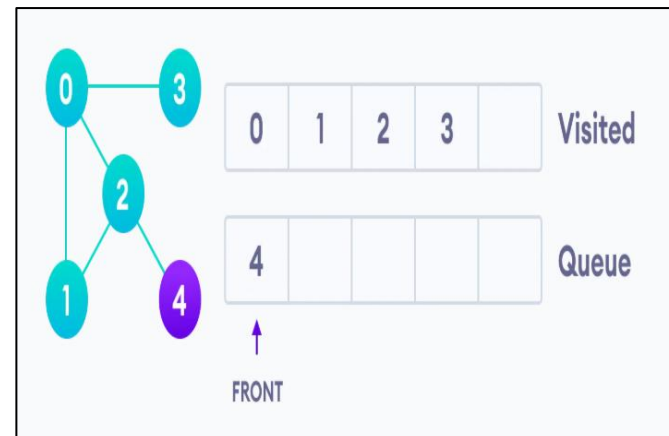
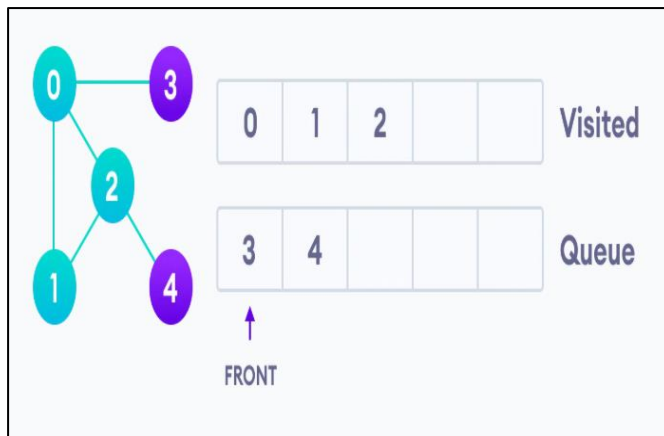
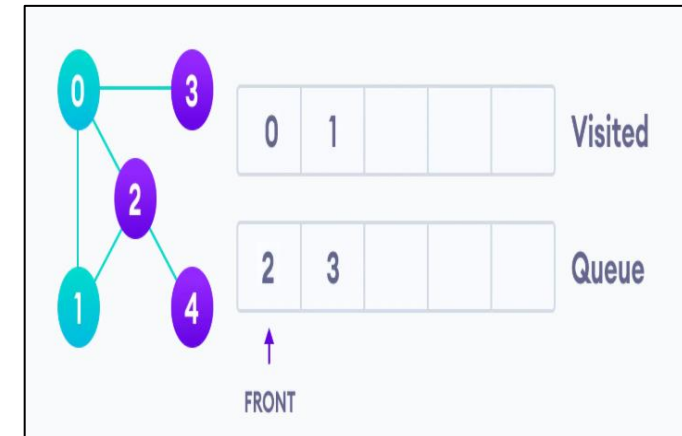
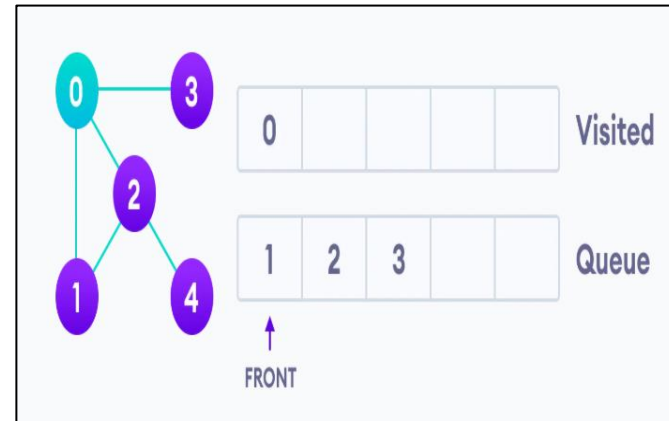
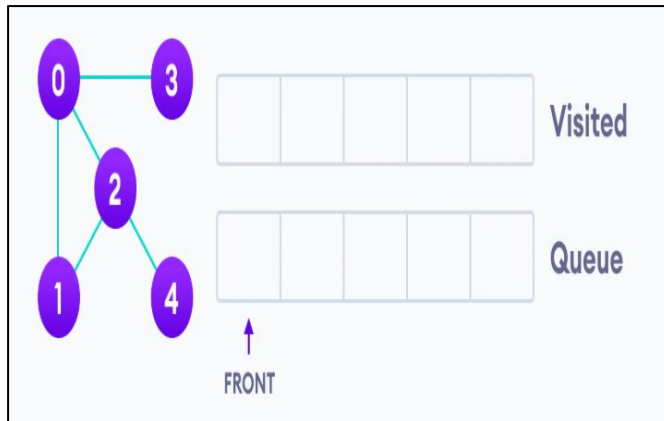
What is BFS?

- BFS is an uninformed search algorithm.
- Explores nodes level by level and left to right.
- All nodes at the current depth are visited before moving to the next level.



Algorithm Steps

- 1 Enqueue the start node:** Put starting node in the queue
- 2 Dequeue and visit:** Take the front item from the queue and add it into the visited list
- 3 Enqueue unvisited neighbors:** Find adjacent nodes that are not visited and add them in the queue
- 4 Repeat until the queue is empty:** Keep repeating step 2 and 3 until the queue is empty



Example of BFS

Now that we've covered BFS, let's move towards practical code-based example

BFS

Breadth First Search on nodes A, B, C, D, E. Starting node is A.

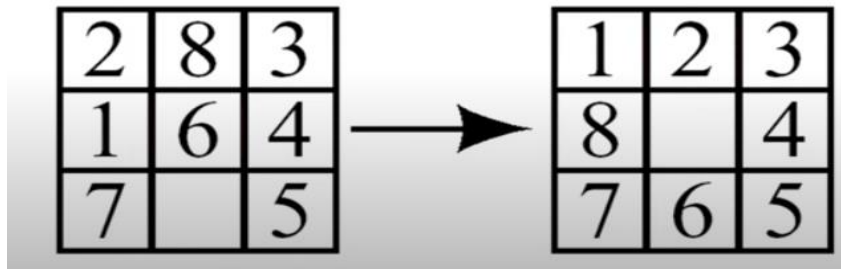


```
graph = { 0: [1, 2], 1: [0, 3, 4], 2: [0], 3: [1], 4: [1, 2] }  
  
def bfs(graph, start):  
    visited = []  
    queue = [start]  
    while queue:  
        node = queue.pop(0)  
        if node not in visited:  
            visited.append(node)  
            for neighbor in graph[node]:  
                if neighbor not in visited:  
                    queue.append(neighbor)  
    return visited  
  
result = bfs(graph, 0)  
print("BFS Traversal Order:", result)
```

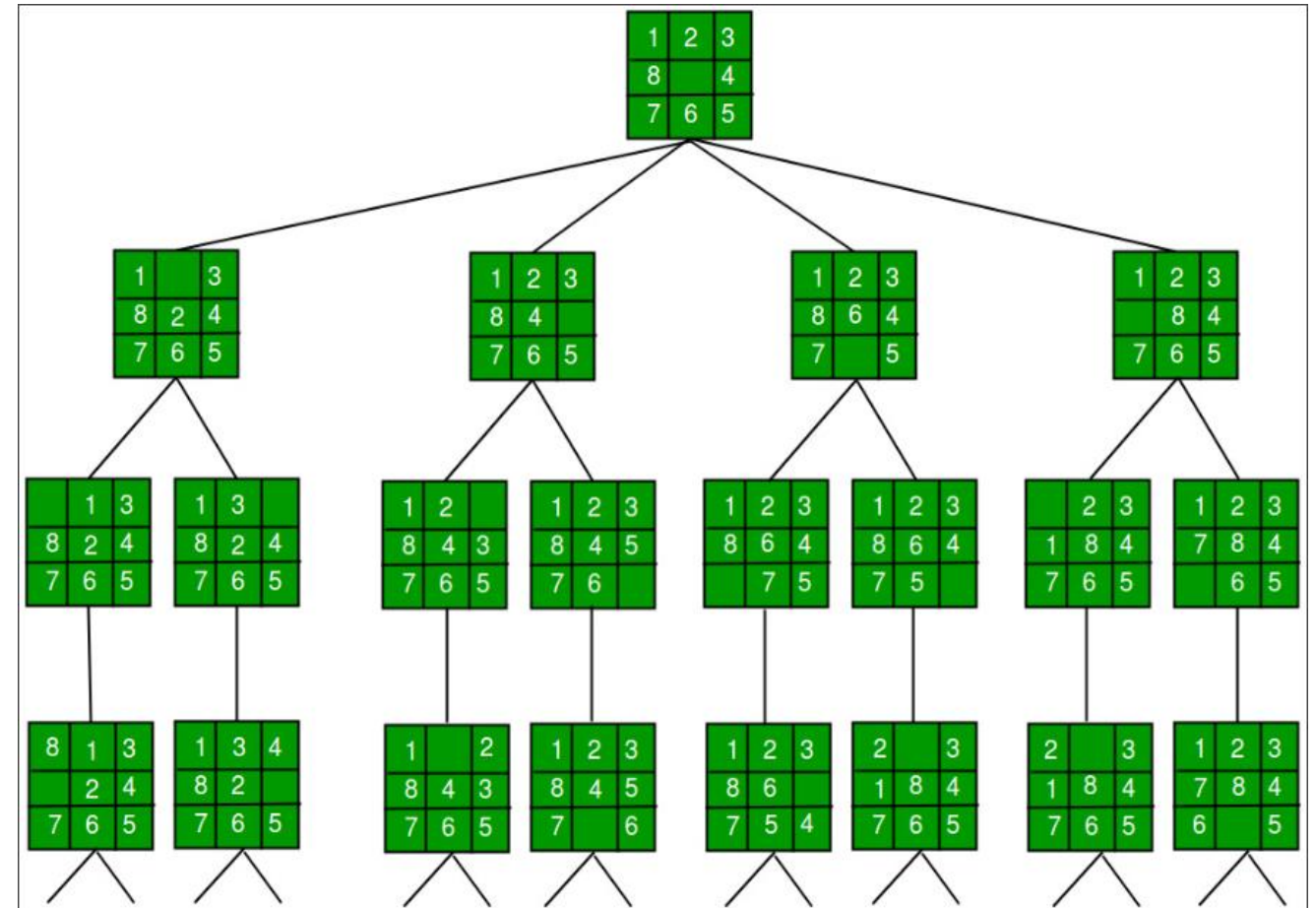

Lab Task: Breadth First Search (Uninformed Search)

- **Expand the Graph:** Add 3 more nodes (5, 6, 7) to the graph dictionary and connect them to existing nodes. Run `bfs()` again starting from 0 — does it still visit every node correctly?
- **Count Nodes Checked:** Add a counter inside the `bfs()` function that increases every time a node is popped from the queue. Print this count along with the traversal — does your bigger graph make BFS check more nodes?
- **Change Starting Point:** Run `bfs()` starting from node 3 instead of 0. Compare the traversal order to `bfs(0)` — why does the order change even though the graph is the same?

8-Puzzle Problem



1 2 3
4 5 6
7 8 0 ← Here, '0' means the empty space



Expansion Tree of 8-Puzzle Problem

Thank You

Questions or discussions are welcomed!